Capítulo 6

Datos estructurados : Arreglos

1. Introducción a datos estructurados

A continuación se presenta dos ejemplos de problemas donde analizaremos la representación de la información involucrada.

1) Se tienen 18 puntos en el plano que corresponden a la relación (x , F(x)), dichos datos serán utilizados para obtener aproximaciones por medio de tres métodos de aproximación de funciones (como Lagrange, Mínimos Cuadrados y Splines Cúbicos)

X	**0.9**	**1.3**	**1.9**	**2.1**	**2.6**	**3.0**	**3.9**	**4.4**	**4.7**	**5.0**	**6.0**	**7.0**	**8.0**	**9.2**	**10.5**	**11.3**	**11.6**	**12.0**
F(x)	**1.3**	**1.5**	**1.85**	**2.1**	**2.6**	**2.7**	**2.4**	**2.15**	**2.05**	**2.1**	**2.25**	**2.3**	**2.25**	**1.95**	**1.4**	**0.9**	**0.7**	**0.6**

2) La siguiente tabla presenta datos de un conjunto de pacientes, sometidos a diferentes tratamientos (codificados con las letras A, B, C, D, E):

Apellido y Nombre	**Edad**	**Peso**	**Tratamiento**
MARTINEZ Juan	40	61.2	D
GONZALEZ Pedro	18	49.5	A
ALONSO Maria	35	55.0	D
ALONSO Marcelo	20	73.1	C

Se desea tener un listado con los datos de los pacientes mayores de 20 años y además el peso promedio de los pacientes sometidos al tratamiento D.

Los dos problemas previos involucran conjuntos de datos asociados entre si. Si bien estos datos pueden ser almacenados en variables simples, esta solución no facilitaría el procesamiento de la información. Además no es una solución de carácter general (la cantidad de información es variable). Una buena solución es almacenar la información en **variables estructuradas.**

- Un tipo estructurado de dato difiere de un tipo simple en que las variables de un tipo estructurado tienen más de un componente.
- Una estructura de datos es un conjunto de datos reunidos bajo un **único** nombre colectivo (su identificador).
- Un <u>tipo de dato estructurado</u> se caracteriza por el(los) tipo(s) de sus componentes y, sobre todo, por el *método de estructuración*. Cada uno de estos métodos es una opción que puede manejar el programador a través de *la sección TYPE*.
- El usuario tiene que analizar la combinación de datos simples para formar *estructuras compuestas* que <u>*representen apropiadamente*</u> la información a procesar; además, debe estudiar como *extraer* y *colocar* componentes específicos en dicha estructura.

Las estructuras que estudiaremos son: arreglos, registros (capítulo 8).

2. Arreglos

Un **arreglo** es una colección ordenada de variables todas las cuales tienen el mismo tipo (esto es, tienen un tipo homogéneo). Cada uno de los elementos de la colección se llama **componente** del arreglo.

Veamos las propiedades de los arreglos:

1) Todas las componentes de un arreglo poseen el mismo tipo de dato.

2) Cada componente de un arreglo se denota explícitamente, y es accedida, directamente, mencionando el nombre del arreglo seguido de una expresión ordinal (entero, caracter) encerrada entre corchetes, a la que llamaremos **índice del arreglo** (o simplemente **subíndice**).

3) Los arreglos poseen **dimensión**, y esta determina la cantidad de índices del arreglo. Un arreglo de dimensión 1 utiliza un índice y representa una columna de datos (arreglos unidimensionales); un arreglo de dimensión 2 utiliza dos índices y representa una tabla de datos dispuesta en filas y columnas (arreglos bi-dimensionales), un arreglo de dimensión 3 (tres índices) representa una tabla de datos en múltiples páginas, etc.

4) Cada dimensión de los arreglos tiene un límite inferior y otro superior. Estos límites determinan la **extensión** de los valores que son usados como subíndices para la dimensión.

5) El **tamaño** de un arreglo (número de componentes) se indica cuando se define el arreglo y queda invariable a partir de ese momento. El cálculo del número de componentes se logra multiplicando entre si la cantidad de elementos que el arreglo posee en cada dimensión.

2.1 Arreglos unidimensionales o Vectores

Veamos los datos del primer problema donde figuran valores de puntos del plano:

X	0.9	1.3	1.9	2.1	2.6	3.0	3.9	4.4	4.7	5.0	6.0	7.0	8.0	9.2	10.5	11.3	11.6	12.0
F(x)	1.3	1.5	1.85	2.1	2.6	2.7	2.4	2.15	2.05	2.1	2.25	2.3	2.25	1.95	1.4	0.9	0.7	0.6

Una posible representación de estos datos consiste en almacenar las abscisas X en una estructura de arreglo unidimensional (también conocido como **vector**):

0.9	1.3	1.9	2.1	2.6	3.0	3.9	4.4	4.7	5.0	6.0	7.0	8.0	9.2	10.5	11.3	11.6	12.0

X

Y en otra estructura los valores de Y =F(x)

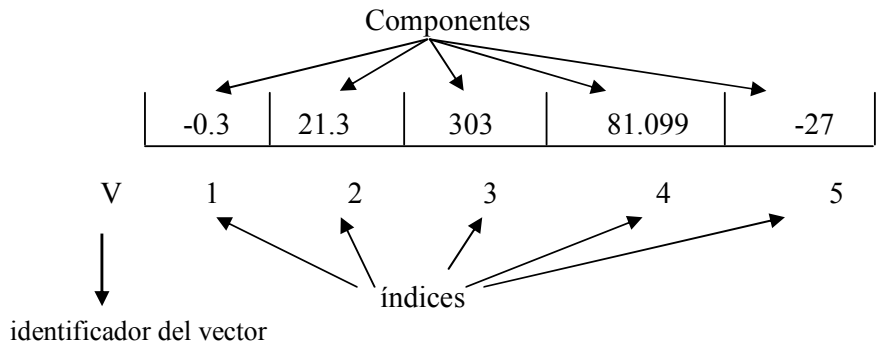
1.3	1.5	1.85	2.1	2.6	2.7	2.4	2.15	2.05	2.1	2.25	2.3	2.25	1.95	1.4	0.9	0.7	0.6

Y

Para procesar esta información ha sido almacenada en dos vectores.
A continuación se presenta un ejemplo sencillo que muestra las características y aspectos a considerar

para trabajar con arreglos unidimensionales.

Hemos almacenado cinco datos reales (con variables simples se debía usar 5 identificadores de variable simple para tenerlos todos en forma simultánea en memoria) en un vector identificado como V**:**



Veamos las propiedades de V:

1) Todas las componentes de V poseen el **tipo de dato** real.

2) Cada componente de V es accedida, directamente, mencionando el identificador del vector seguido del subíndice (expresión ordinal encerrada entre corchetes).
   Por ejemplo, V[4] identifica el cuarto elemento del vector y representa al valor 81.099.
   Los subíndices son computables. El tiempo que se requiere para seleccionar (extraer) un componente no depende del valor del selector (o índice); por lo que el arreglo se denomina *estructura de acceso aleatorio.*

3) V es de una dimensión (**unidimensional**) y por lo tanto posee un índice.

4) La dimensión de V tiene como límite inferior 1 (uno) y como límite superior 5 (cinco). Por lo tanto la extensión de V es 1, 2, 3, 4 y 5.

5) El **tamaño** de V es 5 (esto es, el número de componentes).

Las dos operaciones básicas con arreglos son extracción y almacenamiento.

Extracción: cada componente se denota explícitamente y se lo extrae directamente mediante el identificador del arreglo seguido por el índice entre corchetes (como se describe en 2).

Almacenamiento: se realiza mediante la asignación o lectura.
Por ejemplo V[4]:= -101.2 modifica el contenido del cuarto componente.

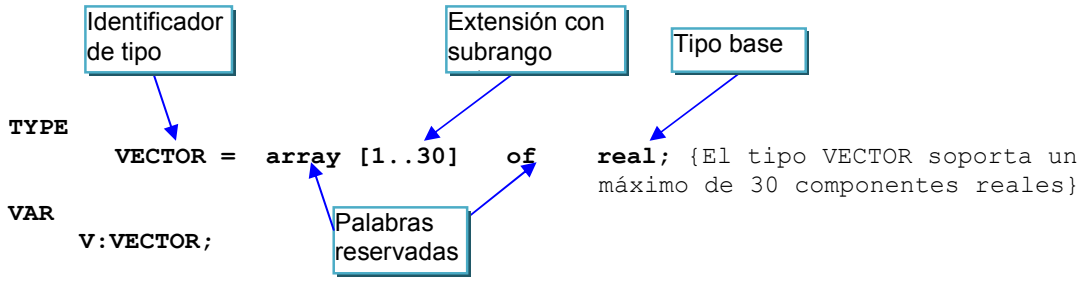
Notemos que un vector tiene asociado dos tipos de datos:

a. Tipo **base** (o tipo **componente)**: es el tipo de los componentes (o también denominados elementos).
b. Tipo **índice**: es el tipo de los valores de la extensión. El tipo índice puede ser entero, carácter, (también subrangos de ambos tipos), y booleano. No se acepta índices de tipo real o string.

En la sección TYPE declaramos el **identificador de tipo** (creado por el programador), la palabra reservada **array**, un subrango para indicar los límites de la extensión de los valores que serán usados

como subíndices para la única dimensión y el tipo base (el tipo de las componentes). Además de estas especificaciones, la declaración del arreglo provoca la reserva de espacio en memoria para todas las componentes.

Veamos la sintaxis:



La variable V por pertenecer al tipo VECTOR tiene reservada en memoria hasta 30 componentes reales. El programador puede decidir qué cantidad de componentes usar (esta puede variar desde 1 hasta 30).

Otros ejemplos de declaración de tipos:

```
CONST
     MaxDim=30;
TYPE
      Estacion=1..4;{1:primavera;2:verano;3:otoño;4:invierno}
      TipoVector=array[1..MaxDim] of real;
      Periodo=array[Estacion] of string[15];
      Vec=array['A'..'Z'] of byte;
VAR
  V:TipoVector; {Esta variable corresponde al ejemplo anterior}
  P:Periodo;
  W:Vec;
  I:Byte;
  C:char;
BEGIN
      V[2]:= 21.3;
      P[3]:='21/03 - 20/07';
      W['B']:= 0;
      W['I']:=45;
      I:=5;
      V[I*3]:=V[I]/V[I-3];
      C:= 'Q';
      W[C]:= 255;
      .........
```

E1) Leer y escribir un vector de N componentes reales, se utiliza un procedimiento para cada tarea. Notar que el tipo del arreglo (TV) permite almacenar hasta 50 números reales, por lo tanto se utiliza una variable entera(N) para almacenar la cantidad exacta de valores ingresados.

```
PROGRAM LeeEscribe;


TYPE
  TV= array [1..50] of real;
VAR
```

```
   N:Byte;
   Vec:TV;

PROCEDURE LeeVector(Var V: TV; Var N:byte);
Var
  i:byte;
begin
  writeln('Cantidad de números reales a leer <=50');
  readln(N);
  for i:=1 to N do
      begin
        writeln('ingrese el elemento', i);  readln(V[i]);
      end;
end;

PROCEDURE EscVector(V: TV; N:byte);
Var
  i:byte;
begin
 for i:=1 to N do
     write(V[i]:5);
end;

BEGIN
   LeeVector(Vec, N);
   EscVector(Vec, N);
END.
```

E2) Sea A un arreglo unidimensional de N elementos enteros, que se ingresa desde teclado. Desarrollar en un único programa Pascal que permita mediante el uso de un menú, seleccionar una de las opciones siguientes:

a. Calcular y mostrar la suma de todos los elementos de A
b. Construir un arreglo B que sea A permutado
c. Construir un arreglo C con los valores impares de A
d. Asignar el valor cero a todos los elementos de A de índice impar.

Imprimir los vectores A (original y modificado), B y C cuando corresponda.

```
PROGRAM E2;
 TYPE
        TVector=array[1..100] of integer;
 VAR
        A,B,C:TVector;
        N,M:integer;
        Opcion:char;

PROCEDURE Menu(Var Op:char);
  Begin
     Writeln('Elija una de las opciones');
     Writeln('a - Suma de  todos los elementos');
     Writeln('b - Permutar los elementos');
     Writeln('c – Valores impares');
     Writeln('d - Asignar cero a las componentes de indice impar');
     Writeln('f - finalizar');
     Readln(Op);
  End;
```

```
PROCEDURE LeeVector(Var A: TVector; Var N:byte);
Var
   i:byte;
begin
  Write('Ingrese la cantidad de numeros');
  Readln(N);
  For i := 1 to N do
        read(A[i]);


end;

PROCEDURE EscVector(V: TVector; N:byte);
Var
  i:byte;
begin
   for i:=1 to N do
        write(V[i]:5);
end;

FUNCTION Suma (A: Tvector; N:byte): longint;
Var
  i:byte;
  S: longint;
begin
  S:=0;
  For i:= 1 to N do
     S:= S + A[i];
  Suma:= S;
End;

PROCEDURE Permuta(A: Tvector; N:byte; Var B: TVector)
   Var
    i: byte;
   Begin
     for i:=1 to N  do
        B[i]:=A[N-i+1];
  End;

PROCEDURE Impares(A: Tvector; N:byte; Var C: Tvector; Var M: byte)
   Var
    i: byte;
   Begin
   M:=0;
   for i:=1 to N  do
     if A[i] mod 2 <> 0 then
       begin
         M:=M+1;
         C[M]:=A[i];
       End;
  End;
```

```
PROCEDURE AsignaCero(Var A: Tvector; N:byte)
  Var
   i: byte;
  Begin
  i:=1;
  WHILE i<=N do
    Begin
      A[i]:=0; i:=i+2
    end
  end;

BEGIN {programa principal}
  LeeVector(A,N);
  EscVector(A,N);
  Repeat
      Menu(Opcion);
      CASE Opcion OF
            'a': Writeln('La suma es  ', Suma(A,N));
            'b': begin
                   Permutar(A, B, N); EscVector(B,N);
                 end;
            'c': begin
                   Impares(A,N,C,M);  EscVector(C,M);
                 end;
            'd': begin
                   AsignaCero(A,N);    EscVector(A,N);
                 end;
      END; {case}
 Until Opcion = 'f'
END.
```

2.1.1 Vectores paralelos

Retomamos el ejemplo de la introducción donde se debe almacenar los diferentes atributos (características) de un conjunto de datos.
La siguiente tabla presenta datos de un conjunto de pacientes, sometidos a diferentes tratamientos (codificados con las letras A, B, C, D, E):

Apellido y Nombre	Edad	Peso	Tratamiento
MARTINEZ Juan	40	61.2	D
GONZALEZ Pedro	18	49.5	A
ALONSO Maria	35	55.0	D
ALONSO Marcelo	20	73.1	C

Cada atributo tiene un tipo diferente, por lo tanto definimos un arreglo para cada uno.

AyN	Edad	Peso	Trat
(vector de string)	(vector de byte)	(vector de real)	(vector de char)

Para mantener la vinculación correcta entre los atributos de cada paciente (AyN, Edad, Peso, Tratamiento) deben asociarse los elementos de los cuatro vectores utilizando la misma posición de índice. Por ejemplo las características del primer paciente (Martinez, Juan) están representadas en NyA[1], Edad[1, Peso[1] y Tratamiento[1] . Igualmente ocurre con el resto de los pacientes. Esta

representación, en **vectores paralelos,** consiste en almacenar y recuperar los ítems de información asociada en **componentes del mismo índice** en cada uno de los vectores paralelos.

E3) Se desea tener un listado de los apellidos y tratamiento de los pacientes mayores de 20 años, y además el peso promedio de los pacientes sometidos al tratamiento D.

```
PROGRAM E3;
TYPE
   TVS=array[1..100] of string[25];
   TVB=array[1..100] of byte;
   TVR=array[1..100] of real;
   TVC=array[1..100] of char;

VAR
  AyN:TVS;
  Edad:TVB;
  Peso:TVR;
  Trat:TVC;
  N:byte;
  P:real;

PROCEDURE LeeVectores(Var AyN:TVS;Var Edad:TVB;Var Peso:TVR;Var Trat:TVC;
Var N:word);
Var
  i: word;
Begin
 Write('Cant. de pacientes');    Readln(N);
 For i:=1 to N do
    BEGIN
       Write('Paciente ',i,' Apellido y Nombre');readln(AyN[i]);
       Write('Edad ');  Readln(Edad[i]);
       Write('Peso ');  Readln(Peso[i]);
       Write( 'Tratamiento '); Readln(Trat[i]);
    End;
End;

PROCEDURE Listado(AyN:TVS; Edad:TVB; Trat: TVC;  N:word);
Var
  i: word;
Begin
 Write('Apellido, Nombre y tratamiento de los mayores de 20 años');
 For i:=1 to N do
    If Edad[i] > 20 then
       Writeln(AyN[i], Trat[i]:8);
End;

FUNCTION Promedio(Peso:TVR Trat: TVC;  N:word):real;
Var
  I, Cont: word;
  S:real;
Begin
 S:=0;  Cont:=0;
 For i:=1 to N do
   If Trat[i] ='D' then
       Begin
         Cont:= Cont +1;
         S:=S+ Peso[i];
       End;
 If Cont <> 0 then
   Promedio:= S/Cont
 Else
   Promedio:=0
End;
```

```
BEGIN {programa principal}
  LeeVectores(AyN, Edad, Peso, Trat, N);
  Listado(AyN, Edad, Trat, N);
  P:= Promedio(Peso, Trat, N);
  If P =0 then
    Writeln('no hay pacientes con tratamiento D')
  Else
    Writeln('el promedio de peso con tratamiento D es', P:8:2)
END.
```

2.1.2 Búsqueda de un determinado valor en un arreglo

Los algoritmos de búsqueda están condicionados por la distribución de los valores en el arreglo, es decir si estos están ordenados o no lo están.
En ambos casos se utiliza el ciclo condicional WHILE, que permite iterar mientras el/los índice/s estén en el rango esperado y el elemento indexado no es el buscado.

Sea TV un tipo vector de reales y X un valor real a buscar.

a) Función entera que busca un valor X en un arreglo V de N elementos desordenados, devuelve la posición donde lo encuentra o N+1 si no está en el arreglo. Se avanza secuencialmente en el arreglo mientras el índice este en el rango de 1 a N y el elemento buscado no coincide con el de la posición del índice, el ciclo se detiene cuando encuentra el elemento o cuando el índice supera N.

```
FUNCTION BuscaSecuencial ( V:TV; N:byte; X:integer ):byte;
Var
  i: byte;
Begin
  i:=1:
  While (i<= N) and (X <> V[i]) do
       i:= i+1;
  BuscaSecuencial := i;
End;
```

b) Función booleana que busca un valor X en un arreglo V de N elementos ordenados, devuelve TRUE si lo encuentra y FALSE en caso contrario. El índice no avanza secuencialmente sino que se calcula como el Medio de un intervalo [Pri, Ult] que se va redefiniendo de acuerdo a la relación del valor buscado y el elemento en la posición Medio. La búsqueda finaliza cuando lo encuentra o no puede redefinir un nuevo intervalo (Pri=Ult).

```
FUNCTION BusquedaBinaria ( V:TV; N:byte; x:integer ):boolean;
Var
    Medio, Pri, Ult: byte;
Begin
  Pri := 1;
  Ult := N;
  Medio:=(Pri + Ult) DIV 2:
  While (Pri <Ult) and (x <> V[ Medio ]) do
       Begin
         If  x < V[ Medio ] then
             Ult := Medio - 1
         Else
              Pri:= Medio + 1;
         Medio:=(Pri + Ult) DIV 2:
       End;
```

```
  BusquedaBinaria:=  x = V[ Medio]
End;
```

Este método es más rápido que la búsqueda secuencial, pues en cada iteración reduce a la mitad el espacio de búsqueda, descartando la mitad donde se sabe que no se encontrará el valor buscado.

E5) Procedimiento que a partir de un vector A de N elementos enteros ordenados, generar y devuelve un nuevo arreglo B donde se han eliminado los elementos repetidos (consecutivos iguales**)**. Notar que si el vector no estuviera ordenado, el algoritmo de eliminación de valores repetidos sería más complejo.

```
program E5;
TYPE
  TV=array[1..100] of integer;
……................
PROCEDURE EliminaRepetidos(A:TV; N: byte; Var B:TV; Var M:Byte);
Var
  I:byte;
begin
  M:=1;
  B[1]:=A[1];
  for i:=2 to N do
     if A[i]<> B[M] then
          begin
            M:=M+1;
            B[M]:=A[i];
          end;
end;
……................
```

2.1.3 Métodos de ordenamiento

El propósito principal de un ordenamiento es el de **facilitar las búsquedas** de los miembros del conjunto ordenado (numérico, alfabético, ascendente o descendente). Además como se ve en el ejemplo E6, el orden de los datos condiciona el algoritmo que opera sobre los mismos.

Ejemplos: guía telefónica, padrón electoral, bibliotecas, diccionarios, etc.
Cuando se menciona "ordenado", por defecto se entiende en forma ascendente.

Para obtener ese orden se emplean técnicas que comparan y mueven los datos, a continuación se describe el método por Intercambio.

Método de ordenamiento ascendente por intercambio. El procedimiento Ordena recibe un arreglo desordenado y lo devuelve ordenado. El método consiste en ir colocando en forma progresiva y ascendente los valores mínimos a la izquierda, para cada posición el mínimo se elige comparándolo contra todos los elementos restantes del vector e intercambiándolos.

```
TYPE
  TV=array[1..100] of integer;
  PROCEDURE Ordena(Var V:TV; N:byte);
  Var
      i,k:byte;  aux:integer;
    Begin
      For k:=1 to N-1 do {N-1 pasadas}
          For i:=k+1 to N do  {elementos de la pasada  k-esima}
              If  V[k]>V[i] then
                  begin
                    Aux:=V[k];  V[k]:=V[i]; V[i]:=aux
                  End
```

```
End;
```

A continuación se presenta una prueba de escritorio para ordenar (en forma ascendente) un vector de 5 componentes: [70,12,4,210,15]:

N	k	i	V[k]>V[i]	aux	V
5					[70, 12, 4, 210,15]
	1				[**70**, 12, 4, 210,15]
		2	True (70>12)	70	[**12**,70,4,210,15]
		3	True (12>4)	12	[**4**,70,12,210,15]
		4	False (4<210)		[**4**,70,12,210,15]
		5	False (4<15)		[**4**,70,12,210,15]
	2				[4,**70**,12,210,15]
		3	True (70>12)	70	[4,**12**,70,210,15]
		4	False (12<210)		[4,**12**,70,210,15]
		5	False (12<15)		[4,**12**,70,210,15
	3				[4,12,**70,**210,15]
		4	False (70<210)		[4,12,**70**,210,15]
		5	True (70>15)	70	[4,12,**15**,210,70]
	4				[4,12,15,**210**,70]
		5	True (210>70)		[4,12,15,**70**,210]

Este algoritmo es sencillo, pero ineficiente y lento. Para una lista de N números, el proceso de ordenación requiere (N-1) pasadas y el número de comparaciones se refleja en la tabla siguiente:

Pasada	Comparaciones
1	N-1
2	N-2
3	N-3
……..	
N-1	1

El número total de comparaciones es:
$1 + 2 + 3 + \ldots + (N-3) + (N-2) + (N-1) = ½ (N^2-N)$

En el peor de los casos, cuando el vector está invertido (ordenado en orden descendente) el número de comparaciones es función de N^2. En el mejor de los casos, el número de intercambios depende de N.
Requiere un excesivo tiempo de ejecución y no detecta si existe un orden total o parcial que evita operaciones innecesarias. Existen otros métodos más rápidos que aceleran el proceso de ordenación y evitan comparaciones inútiles como el de Inserción, Burbuja mejorado, Shell, Quick Sort, etc.

2.2 Arreglos bi-dimensionales (o matrices)

Si se desea almacenar y procesar una matriz algebraica, por ejemplo:

$$A = \begin{bmatrix} 12 & 1002 & -9.9 \\ 1.2 & 76.7 & 9.9 \\ 34.09 & 112 & 34.1 \\ 15 & 1 & -0.92 \end{bmatrix}$$

se puede representar con un arreglo bidimensional (o matriz).

Matriz A	Columna 1	Columna 2	Columna 3
Fila 1	12	1002	-9.9
Fila 2	1.2	76.7	9.9
Fila 3	34.09	112	34.1
Fila 4	15	1-0.92	

A[2,1] A[4,2] A[3,3]

En una estructura de arreglo bi-dimensional, encontramos datos almacenados en dos dimensiones según Filas y Columnas. En el ejemplo tenemos 4 filas y 3 columnas. Cada uno de los 12 elementos, se extraen invocando el nombre de la variable de tipo array seguido por fila y columna entre corchetes.

Veamos las propiedades de A:

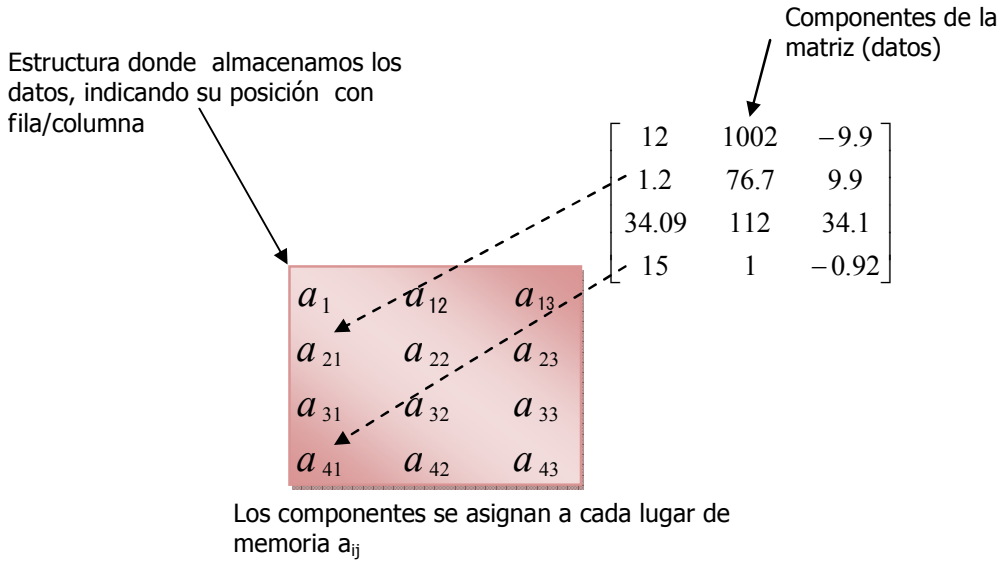
1. Todas las componentes de A poseen el **tipo de dato** real.

2. A es de dos dimensiones (**bi-dimensional**) y por lo tanto posee dos índices.

3. La primera dimensión de A (índice de fila) toma valores en el rango 1..4; la segunda dimensión (índice de columna) toma valores en el rango 1..3.

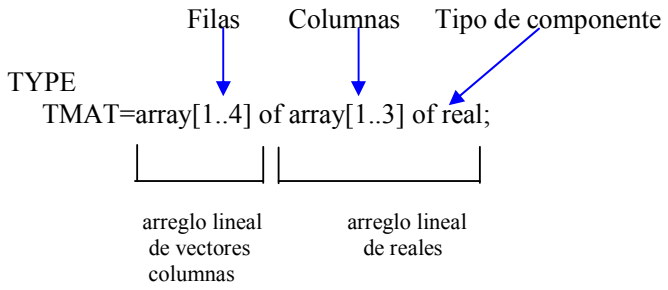
4. El **tamaño** de A es 12 (esto es, el número de componentes). Para conocer la cantidad de elementos se puede calcular:
   Cantidad de filas: cota superior - cota inferior + 1 (4-1+ 1= 4)
   Cantidad de columnas: cota superior - cota inferior +1 (3-1 +1= 3)
   Tamaño de la matriz: cantidad de filas * cantidad de columnas (12)

Cada componente de A es accedida, directamente, mencionando el nombre del arreglo seguido de dos expresiones encerradas entre corchetes, llamadas **subíndices de la matriz.**
Por ejemplo, A[2,1] identifica el elemento de la matriz que se encuentra en la fila 2 y columna 1 y contiene el valor 1.2.
A continuación vemos una representación gráfica de la estructura bi-dimensional (grisada) y los datos que serán almacenados en ella:



2.2.1 Declaración de una matriz

Para declarar datos estructurados como matrices recurrimos a la sección TYPE. Veamos una declaración para la matriz A, creando el tipo MAT.
En la primera declaración se expresa a la matriz como un arreglo unidimensional de columnas (y cada columna en un arreglo unidimensional de reales):



Sin embargo, resulta conveniente usar la siguiente notación, mas compacta:

```
TYPE
       TMAT=  array[1..4,1..3] of real;
                 Indices      componentes

           TM = array[-2..2,0..7] of integer;
Var
   A:TMAT;
   B:TM;
   i,j:integer;
Begin
    A[2,1]:=1.2;
    A[3,3]:=34.1;
     i:=1;    j:=1;
    A[i,j]:=12;
    A[i+2,j+1]:=B[-2,0]*B[-1,2]/5;
  ..............
```

La extensión para la primera dimensión de las variables del TM es -2,-1,0,1,2. Para la segunda dimensión tenemos como extensión 0,1,2,3,4,5,6 y 7. El tamaño es de 40 componentes enteros.
Cantidad de filas:2-(-2)+1= 5
Cantidad de columnas: 7-0 +1=8

2.2.2 Lectura de una matriz por filas

En la memoria se almacenan los datos en forma contigua (una dimensión). En el caso de un arreglo bi-dimensional se almacenan las filas contiguas (1ra fila, 2da fila, etc.), pero se puede decidir el orden en el que ingresaran los datos: por filas o por columnas.

```
TYPE
    TM=array[1..10,1..10] of real;

PROCEDURE LeeXFilas(Var A:TM; Var N,M:byte);
Var
  i, j:byte;
Begin
  Write('Ingrese cantidad de filas <=10');    readln(N);
  Write('Ingrese cantidad de columnas <=10'); readln(M);
  For i:=1 to N do  {fila i-esima}
      For j:=1 to M do
          Begin
            Write('A(',i,',',j,')='); readln(A[i,j])
          End;
 End;
```

A continuación un procedimiento que escribe una matriz en formato matricial

```
PROCEDURE EscribeMat( A:TM;  N,M:byte);
Var
  i j:byte;
Begin
  For i:=1 to N do        {para cada fila}
      Begin
        For j:=1 to M do  {escribe las columnas en la misma línea}
           write(A[i,j])
        writeln;
      End;              { y luego baja a la fila siguiente}
 End;
```

2.2.3 Procesamiento de los elementos de una matriz, ya sea en su totalidad, en determinada/s fila/s o columna/s de la misma.

E6) El siguiente procedimiento calcula el elemento máximo y el mínimo de una matriz.

```
PROCEDURE MaxMin(A:TM; N, M:byte; Var Max, Min:integer);
Var
  i, j:byte;
begin
Max:=A[1,1];
Min:=A[1,1];
```

```
for i:=1 to N do
  for j:=1 to M do
     begin
        if A[i,j]>Max then
          Max:=A[i,j];
        if A[i,j]<Min then
          Min:=A[i,j];
     End;
End;
```

E7) Intercambiar las filas k y l en la matriz A.
Las filas o columnas de una matriz deben tratarse como vectores, es decir, uno de los índices queda fijo, mientras el otro se modifica. Por ejemplo la fila k-ésima de una matriz A es: A[k,1],A[k,2],.....,A[k,n]; la columna j-ésima es: A[1,j],A[2,j],...,A[n,j]; la diagonal principal: A[1,1], A[2,2], ...,A[n,n].

```
PROCEDURE IntercambiaFilas(Var A:TMatriz; N, M, K, L:byte);
Var
  i, j:byte;
  Aux:real;
begin
IF (K>=1) and (K<=N) and (L>=1) and (L<=N) THEN
    For i:=1 to M do {son M cols}
           Begin
             Aux:=A[K,i];
             A[K,i]:=A[L,i];
             A[L,i]:=Aux
           End
ELSE
     Write(´Filas incorrectas')
End;
```

E8) Detectar si una matriz es triangular inferior, se verifica que los todos los elementos sobre la diagonal principal sean nulos . Se presentan tres opciones de algoritmos, considerando que el último optimiza el tiempo de ejecución (en número de acciones elementales)

La matriz A es **triangular inferior** si $a_{ij} = 0$ para todo i<j $\begin{bmatrix} a_{11} & 0 & 0 & 0 \\ a_{21} & a_{22} & 0 & 0 \\ a_{31} & a_{32} & a_{33} & 0 \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}$

Opción 1)
En este caso SE RECORRE LA MATRIZ COMPLETA (por filas), evaluando cuando i<j. Aun cuando encuentra UN elemento distinto de cero que permitiría concluir que no es triangular, continúa el análisis hasta finalizar el recorrido de la matriz.

```
FUNCTION TriangularInferior1(A:TMatri; N:byte):Boolean;
var
   i, j: byte;
   triangular: boolean;
```

```
begin
triangular:=true;
for i:=1 to N do
   for j:=1 to N do
     if i<j then
       if a[i,j]<>0 then
           triangular:=false;
TriangularInferior1 :=triangular;
End;
```

Opción 2)
La mejora en este caso radica en que ya no se recorre la matriz completa sino SOLO LOS ELEMENTOS EN LOS CUALES i<j. De todos modos, no finaliza la búsqueda en el caso de encontrar un elemento no nulo.

```
FUNCTION TriangularInferior2(A:TMatri; N:byte):Boolean;
var
   i, j: byte;
   triangular: boolean;
Begin
triangular:=true;
for i:= 1 to N-1 do
   for j:= i+1 to N do
       if a[i,j]<>0 then
           triangular:=false ;
TriangularInferior2 :=triangular;
End;
```

Opción 3)
En esta opción se optimiza el recorrido de la matriz, analizando solo los elementos en los que i<j y además se finaliza dicho análisis al detectar UN elemento no nulo que permite arribar a una conclusión.

```
FUNCTION TriangularInferior3(A:TMatri; N:byte):Boolean;
var
   i, j: byte;   triangular: boolean;
begin
i:=1;
while (i <N) and triangular do
   begin
     j:=i+1;
     while  (j <= N) and triangular do
        begin
          if a[i,j]<>0 then
             triangular:=false;
          j:=j+1;
        end;
     i:=i+1;
   end;
TriangularInferior3 :=triangular;
End;
```

2.3 Arreglos n-dimensionales

Se puede especificar arreglos de n índices. En este caso se dice que el arreglo es n-dimensional. Por ejemplo Z[i,j,k] menciona un elemento de un arreglo tri-dimensional (pueden pensarse como grupos de matrices).